HOCHSCHULE MITTWEIDA
University of Applied Sciences

BACHELORARBEIT

Herr Paul Fuchs

Die Downtime-freie Bereitstellung von Microservices in Kubernetes Umgebungen.

2026

Fakultät Angewandte Computer- und Biowissenschaften

BACHELORARBEIT

Die Downtime-freie Bereitstellung von Microservices in Kubernetes Umgebungen.

Autor:
Paul Fuchs

Seminargruppe:
MI20w2-B

Studiengang:
Medieninformatik und
Interaktives Entertainment

Erstprüfer:
Prof. Philipp Klimant

Zweitprüfer:
Dr. Rico Beier-Grunwald

Einreichung:
Mittweida, den 27.03.2026

Bibliografische Angaben

Fuchs, Paul:
Die Downtime-freie Bereitstellung von Microservices in Kubernetes Umgebungen.
49 Seiten, Hochschule Mittweida (FH), Fachbereich Angewandte Computer- und Biowissenschaften, Bachelorarbeit 2026

Abstract

In the current world of software engineering developers are measured by the frequency of software releases while operations have to maintain a stable and reliable system thus creating a disbalance in time-to-market and reliability. While DevOps has the methodology to create collaboration between these teams, Site Reliability Engineering is the practical application of these principles. With the introduction of an error budget teams can balance between frequent releases and reliable systems. This thesis analyses how the consumption of error budget can be reduced by using canary releases instead of rolling updates and further by analysing the canary update with white-box monitoring. This study shows that 28% of error budget could be saved by introducing canary releases and an additional 23% of error budget was saved by reducing the detection time using white box monitoring during injection of a failure rate. Injecting waiting time to simulate long answer times this study showed that 23% of error budget could be saved with canary releases and an additional 2% were saved by faster detection times using white-box monitoring.

Zusammenfassung

In der aktuellen Welt der Softwareentwicklung werden Developer-Teams anhand ihrer Veröffentlichungsfrequenz gemessen, während operative Teams nach der Systemstabilität und -verlässlichkeit beurteilt werden. Das erschafft die Notwendigkeit eines Gleichgewichts von frequenten Veröffentlichungen und stabilen Systemen. Mit der Einführung eines Fehlerbudgets kann die Balance dessen gehalten werden. Diese Arbeit untersucht inwiefern eine Verringerung des Fehlerbudgetverbrauchs mithilfe der Canary Veröffentlichungsstrategie gegenüber der Rolling Strategie möglich ist und anschließend inwiefern White-Box Monitoring eine weitere Verringerung des Verbrauchs des Fehlerbudgets durch schnellere Detektionszeit ermöglicht. Die in dieser Arbeit durchgeführten Experimente haben gezeigt, dass die Canary Strategie 28% weniger Fehlerbudget als die Rolling Strategie verbrauchte und sich zusätzlich weitere 23% des Budgets durch eine reduzierte Detektionszeit mittels White-Box Monitoring, bei injizierter Fehlerrate, einsparen ließen. Für eine simuliert längere Antwortzeit konnte die Canary Strategie gegenüber dem Rolling Update 23% des Fehlerbudgets schonen, zusätzlich weitere 2% konnten durch reduzierte Detektionszeit mithilfe von White-Box Monitoring eingespart werden.

Inhaltsverzeichnis

1 Einleitung **1**

2 Theoretische Grundlagen **3**

2.1 Microservice Architektur 3

2.2 Microservice Orchestrierung mit Kubernetes 4

2.3 Deployment Strategien 6

2.3.1 Rolling Updates 6

2.3.2 Canary 6

2.4 Site Reliability Engineering 7

2.4.1 Konzepte und Prinzipien 7

2.4.2 Monitoring und Observability 8

2.5 Downtime 9

3 Konzeption und Methodik **11**

3.1 Fallbeispiel 11

3.2 Systemarchitektur 11

3.3 Konzeptioneller Vergleich der Fehlerauswirkungen bei schrittweisen und vollständigen Software-Bereitstellungen 13

3.4 Parameter zur Analyse 14

3.4.1 Fehlerfreier Zustand des Systems 15

3.4.2 Fehlerszenario A 15

3.4.3 Fehlerszenario B 15

3.4.4 Analyseintervalle 16

4 Ergebnisse 17

4.1 Fehlerszenario A 17

4.1.1 Mittlere Detektionszeit und Mittlere Wiederherstellungszeit 17

4.1.2 Die einzelnen Testläufe im zeitlichen Verlauf 18

4.1.3 Die gemittelten Werte der Deployment Strategien im Vergleich 20

4.2 Fehlerszenario B 20

4.2.1 Mittlere Detektionszeit und Mittlere Wiederherstellungszeit 20

4.2.2 Die einzelnen Testläufe im zeitlichen Verlauf 21

4.2.3 Die gemittelten Werte der Deployment Strategien im Vergleich 23

4.3 Fehlerbudget Verbrauch pro Deployment Strategie 23

4.3.1 Fehlerszenario A 23

4.3.2 Fehlerszenario B 24

5 Diskussion 27

5.1 Interpretation der Ergebnisse 27

5.1.1 Fehlerszenario A 27

5.1.2 Fehlerszenario B 28

5.2 Der erste Testlauf der injizierten Fehlerrate für das Canary Release mit globaler Metrikanalyse 29

5.3 Nicht gemessene Fehler während der ersten Canary Phase 29

5.4 Detektionszeit in Zusammenhang des Fehlerbudgetverbrauchs . . 30

6 Kritische Auseinandersetzung 31

6.1 Empfehlungen für weiterführende Forschungen 32

7 Fazit 33

Abbildungsverzeichnis

2.1 Anfrageverarbeitung von Services an Pods mit Gateway API . . . 5

3.1 Systemarchitektur: EKS Cluster Versuchsaufbau 12

4.1 Summe der Zeit bis zur Detektion und der Zeit bis zur Wiederherstellung für injizierte Fehlerrate 18

4.2 Fehlerrate Rolling Update im Einzelvergleich 18

4.3 Fehlerrate Canary Release globaler Metrikanalyse im Einzelvergleich 19

4.4 Fehlerrate Canary Release isolierter Analysemetrik im Einzelvergleich 19

4.5 Fehlerrate unterschiedlicher Deployment Strategien 20

4.6 Summe der Zeit bis zur Detektion und der Zeit bis zur Wiederherstellung für injizierte Latenz 21

4.7 Latenz Rolling Update im Einzelvergleich 21

4.8 Latenz Canary Release globaler Metrikanalyse im Einzelvergleich 22

4.9 Latenz Canary Release isolierter Analysemetrik im Einzelvergleich 22

4.10 Latenzen unterschiedlicher Deployment Strategien 23

4.11 Verbrauch des Fehlerbudgets unterschiedlicher Deployment Strategien bei injizierter Fehlerrate über die Testdauer 24

4.12 Verbrauch des Fehlerbudgets unterschiedlicher Deployment Strategien bei injizierter Latenz über die Testdauer 25

Abkürzungsverzeichnis

Abkürzung	Bedeutung
SRE	Site Reliability Engineering
SLO	Service Level Objective
SLI	Service Level Indicator
SLA	Service Level Agreement
MTTR	Mean Time To Recover
MTTD	Mean Time To Detect
API	Application Programming Interface
VM	Virtual Machine

Kapitel 1

Einleitung

In der modernen Welt der agilen Softwareentwicklung werden Entwicklerteams daran gemessen, wie schnell sie neue Features in das bestehende System implementieren können (Vgl. [**DoraDevMetrics**]), während operative Teams daran gemessen werden, wie stabil und zuverlässig das System läuft, welches sie operieren (Vgl. [**vangala2025enhancing**]). Hier entsteht die Notwendigkeit eines Gleichgewichts zwischen Entwicklern, die ihre entwickelten Softwareanpassungen möglichst schnell auf die Produktivumgebung ausrollen möchten und den Operativen Teams, die sicherstellen müssen, dass das System weiterhin zuverlässig funktioniert, selbst in modernen DevOps Ansätzen (Vgl. [**TTMReliable**], S. 3245). Mit der Einführung eines Fehlerbudgets wird eine Brücke zwischen schnellen Veröffentlichungen und stabilem System gebaut. Das Fehlerbudget ermöglicht es beiden Teams ein Gleichgewicht in der Erfüllung ihrer Ziele zu finden. Ist das Fehlerbudget ausgereizt, werden keine Veröffentlichungen ausgerollt. (Vgl. [**panda2025benefits**], S. 284) In dieser Arbeit wird quantitativ analysiert, wie der Verbrauch des Fehlerbudgets bei der Bereitstellung neuer Softwareversionen mithilfe reduzierter Detektionszeit und verringertem Nutzerzugriff weiter reduziert werden kann, damit selbst nach einem fehlerhaftem Release weiterhin Fehlerbudget zur Verfügung steht. Dabei wird die Forschungsfrage *In welchem Maße kann durch begrenzten Nutzerzugriff und die Verwendung von White-Box Monitoring der Einfluss fehlerbehafteter Softwareversionen auf ein Fehlerbudget beeinflusst werden?* in den Mittelpunkt der Arbeit gestellt. Nachdem Theoretische Grundlagen und deren Relevanz für diese Arbeit erklärt sind, wird der Versuchsaufbau und die Systemarchitektur erläutert. Anschließend werden die Ergebnisse bezeichnet, woran sich die Interpretation und Diskussion anschließend. Zuletzt wird die For-

schungsfrage beantwortet.

Kapitel 2

Theoretische Grundlagen

2.1 Microservice Architektur

In dieser Arbeit wurde sich für eine Microservice Architektur entschieden, um mithilfe der *Organisierung nach Geschäftsprozessen* einen fehlerinjizierten Service von der Vergleichsgruppe abzugrenzen. Alternativ hätte die Überprüfung auch durch verschiedene Endpunkte und Funktionen innerhalb desselben Service durchgeführt werden können. James Lewis und Martin Fowler beschrieben bereits 2014 Microservice Architekturen und grenzten dabei den Architekturansatz klar von monolithischen Anwendungen ab. Im Unterschied zu diesen, beschreiben sie die Microservice Architektur als Ansatz bei dem eine Anwendung aus dem Verbund mehrerer Services besteht, von denen jeder als eigener Prozess läuft und deren Kommunikation über leichtgewichtige Mechanismen, bspw. HTTP-Ressource API's stattfindet. Dabei werden die Services auf Geschäftsprozesse zugeschnitten und lassen sich unabhängig voneinander testen, bereitstellen und skalieren. Weiter beschreiben sie für Microservice Architekturen die folgenden Charakteristika:

- Komponenten durch Services
- Organisierung anhand der Geschäftsprozesse
- Produkte, keine Projekte
- schlaue Endpunkte und Nachrichtendepots
- Dezentrale Steuerung

- Dezentrales Datenmanagement
- Automatisierung der Infrastruktur
- Fehlertoleranz
- Evolutionäres Design

Diese Charakteristika lassen sich Ihrer Aussage nach bei den meisten Microservice Architekturen finden, sind jedoch nicht allgemeingültig oder als Definition zu verstehen. (Vgl. [**Fowler2014**])

2.2 Microservice Orchestrierung mit Kubernetes

82% der Unternehmen, die mit Containern arbeiten, verwenden Kubernetes. Daraus erfolgt, dass Kubernetes zum Standardrepertoire für die Orchestrierung von Container-basierten Applikationen in Produktivumgebungen zählt. (Vgl. [**CNCFSurv**], S. 11) Kubernetes ist eine Open-Source Softwarelösung ist, die containerisierte Applikationen und Services verwaltet. In Produktionsumgebungen sollen die Container verwaltet werden und es muss sichergestellt sein, dass es keine Downtime gibt. Kubernetes ermöglicht die Skalierung und Failover Mechanismen von Applikation. Einige der Hauptaufgaben, die Kubernetes übernimmt, sind die Service Erreichbarkeit und Lastverteilung, die automatisierte Veröffentlichung und Rollbacks und die Selbstheilung. (Vgl. [**K8sOverview**]) Die kleinsten ausrollbaren Einheiten in Kubernetes sind Pods. Diese dienen zur Ausführung der containerisierten Applikation und können von übergeordneten Komponenten verwaltet werden. (Vgl. [**K8sWl**]) Eine dieser übergeordneten Komponenten von Pods sind Services. Sie dienen als nach außen gerichtete Endpunkte und verteilen den ankommenden Datenverkehr auf die ihm zugeordneten Pods. (Vgl. [**K8sSvc**]) Abbildung 2.1 verdeutlicht die Anfrageweiterleitung der nach außen gerichteten Endpunkte von Services an die entsprechenden Pods. So werden Anfragen von außerhalb des Clusters direkt an die jeweiligen Pods zur Verarbeitung weitergeleitet.

Abbildung 2.1: Anfrageverarbeitung von Services an Pods mit Gateway API

2.3 Deployment Strategien

Um Microservices bereitzustellen und währenddessen die Funktionsfähigkeit des Systems nicht zu beeinflussen, haben sich unterschiedliche Strategien entwickelt. Dazu zählen die Strategien Big-Bang (oder Recreate), Rolling, Blue-Green, Canary, A-B Testing und Shadow. (Vgl. [**deployStraDie**]) In dieser Arbeit wird die Rolling Strategie, als Kubernetes Standard Strategie, der komplexeren Canary Strategie gegenübergestellt. Dies soll zeigen, inwiefern der Mehraufwand der Implementierung komplexerer Strategien Vorteile bei der Einsparung des Fehlerbudgets und daraus resultierend, ein verlässlicheres System etabliert werden kann. Von der Gegenüberstellung weiterer Strategien wurde abgesehen, da die Recreate Strategie bedingte Downtime verursacht. (Vgl. [**deployStraDie**], S. 2) A-B Testing wurde nicht in Betracht gezogen, da diese Strategie weniger für infrastrukturelle, sondern für Nutzererfahrung entwickelt wurde. (Vgl. [**deployStraDie**], S. 3) Weiterführend wurde von der Implementierung der Blue-Green- und Shadow Strategie, aufgrund des Mehraufwands für Ressourcen, abgesehen. Für die Implementierung der Rolling- und Canary Strategie wurde sich für Argo-Rollouts entschieden, da sich unter Verwendung eines Service Meshes die prozentuale Lastverteilung realisieren lässt. Alternativ hätten sich die Strategien auch Kubernetes nativ implementieren lassen. Hier hätte die prozentuale Verteilung für das Canary Deployment auf der Replica-Anzahl der Pods basiert.

2.3.1 Rolling Updates

Als Rolling Update wird die Bereitstellungsstrategie bezeichnet, bei der ein Pod mit neuer Softwareversion lauffähig gemacht wird und nachdem die Lauffähigkeit festgestellt ist, wird einer der Pods mit der alten Softwareversion heruntergefahren. Dieser Vorgang wird so lange wiederholt, bis alle Pods auf die neue Softwareversion aktualisiert wurden. (Vgl. [**deployStraDie**], S. 2)

2.3.2 Canary

Die Strategie des Canary Bereitstellungsprozesses wird durch anteilige Zurverfügungstellung definiert. Hierbei wird die neue Softwareversion nur einem Anteil der Nutzer zugänglich gemacht. Dies ermöglicht das Testen der Software unter rea-

ler Produktionslast. Ist validiert, dass die anteilig neu ausgerollte Softwareversion den Bedingungen von tatsächlichen Nutzern genügt, findet eine flächendeckende Aktualisierung der Softwareversion aller Pods statt. (Vgl. [**deployStraDie**], S. 3)

2.4 Site Reliability Engineering

SRE entstand aus der Notwendigkeit komplexe Softwaresysteme verlässlich, skalierbar, verfügbar und effizient zu verwalten. Mit dem SRE Ansatz wird das operative Verwalten von der Reaktion auf Fehler auf eine proaktive Vorsorge der Fehlerverhinderung und Fehlereindämmung gelenkt. (Vgl. [**gopikrishna2025site**], Kap. 1) Im Zuge dieser Arbeit bietet SRE den praktischen Ansatz zur Beantwortung der Forschungsfrage, da mithilfe der Definition von Schwellwerten und der Überwachung des Systems die Basis zur Beantwortung gelegt ist.

2.4.1 Konzepte und Prinzipien

Als Schlüsselkompetenz wird SRE die Fähigkeit zugeordnet, Softwaresysteme verlässlich zu verwalten. (Vgl. [**BenefSRE**], S. 284) Weitere Kernkompetenzen von SRE belaufen sich auf die Skalierbarkeit, die Leistung, sowie die Effizienz des Systems. (Vgl. [**gopikrishna2025site**], S. 3) Für die Überprüfbarkeit der Kernkompetenzen werden SLI's, SLO's und SLA's definiert. SLI's messen konkrete Aspekte, wie Latenz oder Fehlerrate, eines Systems. SLO's werden definiert um einen Wertebereich einzugrenzen, innerhalb dessen sich die SLI's bewegen dürfen. (Vgl. [**gopikrishna2025site**], S. 3) Service Level Agreements werden als Vertragsverbindlichkeiten beschrieben, deren Nicht-Einhaltung meist finanzielle Konsequenzen nach sich trägt. (Vgl. [**GoogleSRE**], S. 39f)

Risiken eingehen

Da die Praxis von SRE maßgeblich durch Google geprägt wurde (Vgl. [**gopikrishna2025site**], S. 4), wird sich an den Praktiken des Unternehmens und deren Zielmaßstab orientiert, wenngleich die Implementierung der Praktiken innerhalb einer Bachelorarbeit nicht der Stärke verschiedener Entwicklungsteams entspricht. Alvidrez und Guliani beschreiben, dass von einem Unternehmen wie Google erwartet würde,

dass die Services zu 100% zuverlässig wären. Doch im Folgenden wird festgestellt, dass ab einem bestimmten Punkt die Erhöhung der Zuverlässigkeit einer Software zur Verschlechterung des Services führt. Damit gehe einher, dass neue Features weniger häufig veröffentlicht werden könnten, während Nutzer den Unterschied von hoher zu extremer Verlässlichkeit kaum spürten. (Vgl. [**GoogleSRE**], S. 25) Im letzten Roth und Quinito beschreiben weiter die Motivation für ein Fehlerbudget. Dabei erklären sie, dass ein Fehlerbudget als objektive Metrik genutzt wird, um einzuschätzen, wie unzuverlässig ein System während einer bestimmten Zeitperiode sein dürfe. Demnach wird der Hauptvorteil eines solchen Fehlerbudgets wird als Gleichgewicht von Innovation und Zuverlässigkeit beschrieben. So lange sich das System innerhalb des Fehlerbudgets bewege, können neue Features veröffentlicht werden, wird das Budget jedoch überschritten, finde innerhalb des definierten Zeitraums keine weitere Veröffentlichung statt. (Vgl. [**GoogleSRE**], S. 33) Mit der Definition des Fehlerbudgets wird in dieser Arbeit die Grenze zwischen Degradierung und Ausfall (siehe Abschnitt *Downtime*) des Systems gesetzt.

Service Level Objectives

Jones et al. (2017) definieren die Terminologien in Zusammenhang mit SLO's. Die SLI's werden als definiertes quantitatives Maß eines Aspekts des Services beschrieben, bspw. Latenz oder Fehlerrate. Service Level Objectives werden als Zielwert oder Wertebereich von Service Level Indicators, bspw. die Fehlerrate ist weniger als ein Prozent, definiert. Service Level Agreements werden als Vertragsverbindlichkeiten beschrieben, deren Nicht-Einhaltung meist finanzielle Konsequenzen nach sich trägt. (Vgl. [**GoogleSREChap4**]) Für diese Arbeit sind die Service Level Indicators (SLI) und Service Level Objectives (SLO's) von Bedeutung, da sie den Rahmen erlaubter bzw. kalkulierter Downtime eingrenzen.

2.4.2 Monitoring und Observability

IEEE definiert einen *Monitor* als "Ein Softwaretool oder Hardwaregerät, das parallel zu einem System oder einer Komponente arbeitet und den Betrieb des Systems oder der Komponente überwacht, aufzeichnet, analysiert oder überprüft." S.49 [**IEEETerms**]

Observability wird als Eigenschaft der Kopplung von Zustand und Ausgabe beschrieben. (Vgl. [**gopal1993modern**], S. 207) Einfacher ausgedrückt zeigt die Observability an, zu welchem Grad ein System überwacht werden kann (Vgl. [**SinaObservAndMon**], S. 37)
Ewaschuk und Beyer beschreiben das Monitoring verteilter Systeme als ”Sammeln, Verarbeiten, Zusammenführen und Anzeigen quantitativer Echtzeitdaten” S.59 ([**GoogleSRE**]) eines Systems. Weiter wird zwischen White-Box Monitoring, dem Auslesen Service-interner Metriken und Black-Box Monitoring, dem außerhalb sichtbaren Verhalten, ”wie ein Nutzer es sehen würde” ([**GoogleSRE**], S. 59) unterschieden. Folgend werden die Signale des Monitoring beschrieben (Vgl. [**GoogleSRE**], S. 60), dazu gehören:

- Latenz
- Datenverkehr
- Fehler
- Sättigung

In dieser Arbeit wird das Monitoring der Latenzen und Fehlerraten verwendet, um die Gegnüberstellung der unterschiedlichen Deployment Strategien zu analysieren und zu bewerten.

2.5 Downtime

Downtime wird als Zeitperiode beschrieben, in der ein Service oder System nicht erreichbar ist. Als Erweiterung dessen gilt Downtime auch als Zeitperiode der nicht-Verfügbarkeit innerhalb vereinbarter Schwellwerte. Zum Begriff der Downtime gibt es weitere Terminologien, die sich in Zusammenhang mit Downtime bringen lassen. Dazu gehören vom (Total-)Ausfall über eine Degradierung bis hin zu Fehlern mit verschiedenen Bezügen zum Begriff Downtime. (Vgl. [**SRESchoolDownt**])
Für diese Arbeit lässt sich Downtime der Zeit zurodnen, die das System außerhalb definierter SLO Schwellwerte nicht funktionsfähig ist. Downtime-Freiheit bezieht sich auf die Abwesenheit der nicht-Funktionsfähigkeit außerhalb der definierten Schwellwerte.

Kapitel 3

Konzeption und Methodik

3.1 Fallbeispiel

Als sinnbildliches Szenario wurde ein Online-Shop gewählt. Die Architektur des Systems besteht aus zwei Microservices, welche jeweils eine Kommunikation mittels REST-API zur Verfügung stellen. Im Beispielszenario werden die Services als Shop- und Transaktions-Service benannt. Zwischen den Services besteht technisch keine Abhängigkeit. Im Microservice für die Transaktionen wird ein simulierter Fehler injiziert, während der Shop-Microservice als Referenz dient, um gegebenenfalls infrastrukturelle Fehler erkennbar zu machen, welche nicht auf die injizierten Fehler zurückzuführen sind. Da es sich hierbei um eine nutzerzentrierte Anwendung handelt, werden zwei Fehlerszenarien überprüft. Einerseits die Injektion einer Fehlerrate, in dem der Service einen internen Serverfehler produziert. Andererseits die Simulation verlängerter Antwortzeit. Es wurde jeweils überprüft, wie lange die jeweilige Analyse benötigt, den Fehler zu erkennen und anschließend den stabilen Zustand widerherzustellen. Weiter wurde berechnet, wie hoch der Verbrauch des Fehlerbudgets innerhalb des getesteten Zeitfensters war.

3.2 Systemarchitektur

Das System wurde in einem AWS EKS Cluster mit zwei m7i-flex.large Nodes aufgesetzt. Die Entscheidung für ein *managed Cluster* liegt in der Einfachheit begründet, es wurde davon abgesehen unterschiedliche VM's als Nodes zu provi-

sionieren. Es wurde sich für die m7i-flex.large Node entschieden, da diese die ressourcenstärksten zur Verfügung stehenden und im AWS-Free-Tier zugänglichen EC2-Instance Types für universelle Zwecke sind. (Vgl. [**AWSInstType**]) Diese wurden jeweils dediziert, einersetis für die Ausführung der Monitoring-Software und Lasttests und andererseits für die Ausführung der zu testenden Services, konfiguriert. Dies verhindert, dass die Messungen durch Konkurrenz um Hardware-Ressourcen beeinflusst werden. Die Wichtigkeit der Trennung liegt darin begründet, dass die Messergebnisse nicht eindeutig auf den injizierten Fehler sondern auf Ressourcen-Konkurrenz zurückgeführt werden könnten. Abbildung 3.1 veranschaulicht diesen Zustand noch einmal tiefergehend. Hinzuzufügen ist bei der Systemarchitektur, dass Cilium als Service-Mesh konfiguriert wurde, um prozentuale Lastverteilung während des Canary Prozesses zu ermöglichen. Weiter hätte dies das Monitoring für die Nutzung der Services von außerhalb des Clusters vereinfacht. Da die Lasttests jedoch über den Cluster-internen DNS aufgelöst wurden, spielt die Implementierung dessen hier keine weitere Rolle. Die interne Cluster DNS Auflösung ist auf eine Verringerung der Kosten zurückzuführen, da AWS Datentransferkosten über öffentliche IP-Adressen berechnet (Vgl. [**AWSBilling**]) Die zu testende Applikation besteht aus zwei JVM-basierten Microservices, wel-

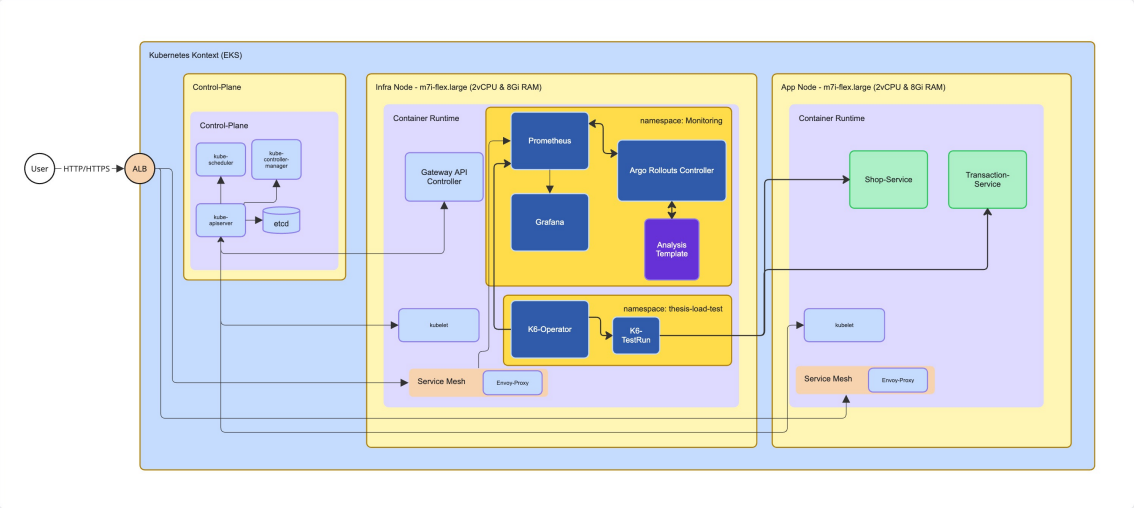

Abbildung 3.1: Systemarchitektur: EKS Cluster Versuchsaufbau

che sinnbildlich einen Online-Shop abbilden. Hierfür wurde, aufgrund bereits bestehender Erfahrungen, als Server-Framework Ktor verwendet. Um Fehler während des Deployment Prozesses zu vermeiden, wurden die Container der Microservices jeweils mit Readiness-Probes und einer Mindest-Ready-Zeit von 20 Sekunden ausgestattet. Dies verhindert, dass die Services Anfragen an Pods weiter-

leiten, die sich noch im Warmup-Prozess befinden. Weiter wurden die Container mit Pre-Stop Hooks von 20 Sekunden versehen, so wird verhindert, dass Anfragen an Pods geleitet werden, die bereits herunterfahren. Um die Erreichbarkeit der Pods sicherzustellen wurden Liveness-Probes implementiert. Diese Applikationen werden auf der Node mit dem Label *role: app-node* gehostet. Die folgenden Software-Tools wurden so konfiguriert, dass sie auf der Node mit dem Label *role: infra-node* ausgeführt werden. Um das Monitoring zu ermöglichen, wurden Prometheus und Grafana im Cluster deployed. Für die Implementierung der jeweiligen Deployment Strategie wurde Argo-Rollouts verwendet. Dies ermöglicht in Verbindung mit Prometheus die SLO-gesteuerte Analyse während des Deployment Vorgangs. Um die Nutzerlast auf den jeweiligen Service zu simulieren wurde K6 mit dem Constant-Arrival-Rate Szenario verwendet. Dies garantiert, dass die Services eine konstant gleichbleibende Last, unabhängig von der Antwortzeit bearbeiten müssen.

3.3 Konzeptioneller Vergleich der Fehlerauswirkungen bei schrittweisen und vollständigen Software-Bereitstellungen

Rein mathematisch lässt sich bereits beweisen, dass Canary-Releases einen Vorteil beim Verbrauch des Fehlerbudgets gegenüber Rolling Updates haben, in dem Sinne, dass bei gleicher Art des Fehlers, Nutzer kontinuierlich gesteigerten Zugriff auf die neue Softwareversion erhalten. (Vgl. [**KhandeCan**], S. 103) Genauer erklärt am Beispiel: Es wird ein Fehler in der Softwareversion ausgerollt, der in zwei Prozent aller Fälle einen internen Serverfehler verursacht. Für Rolling Updates erhalten während und nach der Bereitstellung alle Nutzer Zugang zu der neuen fehlerhaften Version, der Einfluss auf den Verbrauch des Fehlerbudgets spiegelt sich direkt wider. Bei Canary Updates werden während des Bereitstellungsprozesses kontinuierlich mehr Nutzer auf die fehlerhafte Version Zugriff erhalten, demnach spiegelt sich während des Deployment Prozesses der Einfluss auf den Verbrauch des Fehlerbudgets anteilig wider. Erst mit dem Abschluss des Canary Releases spiegelt sich der Einfluss direkt auf das Fehlerbudget wider. Weiter ist herzuleiten, dass die Detektionszeit für den Fehler sinkt, wenn sich die Analyse auf die

Canary-Version beschränkt, anstatt den Fehler im gesamten System zu überprüfen (Vgl. [**islam2024predictive**], S. 236-238). Diese Arbeit analysiert den Faktor, um den die Detektionszeit und der Verbrauch des Fehlerbudgets, bei gleicher zeitlicher Analyseintervallrate zwischen Rolling Update und Canary Release mit globaler und isolierter Aggregation sinkt.

3.4 Parameter zur Analyse

Für die Erhebung der Daten wurden jeweils fünf Testläufe über 30 Minuten absolviert. Es wurden fünf Testläufe gewählt, um eine Reproduzierbarkeit der Versuche sicherzustellen. Es wurde sich für eine Länge des Testlaufs von 30 Minuten entschieden, um den gesamten Canary Prozess innerhalb des Zeitfensters betrachten zu können. Jeder Service war einer Last von 1200 Anfragen pro Sekunde ausgesetzt, bei diesem Wert wurde sich an der Arbeit von Gustafsson und Persson orientiert, die durch 1200 Requests pro Sekunde eine ca. 70% Auslastung ihrer getesteten Server erreichten (Vgl. [**gustafsson2024comparison**], S. 36) Die injizierten Fehler wurden mittels Umgebungsvariablen realisiert. Da Rolling Updates nativ keine Möglichkeit der Analyse bieten, wurde sich dazu entschieden, mithilfe des Alertings von Grafana einen fehlerhaften Zustand des Systems erkennbar zu machen, woraufhin ein manuelles Rollback des Services durchgeführt wurde.
Für die Gewichtung des Datenverkehrs wurden folgende Schritte definiert, nach dem Erhöhen wurde das System für jeweils 5 Minuten überwacht, dies stellt fünf Analysen je erhöhter Gewichtung des Datenverkehr sicher, bevor der nächste Schritt erfolgte:

- 10%
- 50%
- 75%
- 100%

3.4.1 Fehlerfreier Zustand des Systems

Der Zustand der Applikation wurde zuerst ohne injizierte Fehler analysiert, basierend darauf wurde die Definition der SLO's erstellt. Die für diese Arbeit wichtigen Definitionen umfassen:

- eine Verfügbarkeit von 99%
- eine Antwortzeit von 15ms für 95% aller erfolgreichen Anfragen

und beschreiben einen rollierenden Zeitraum von 30 Minuten, im Sinne der Testdauer. Es wurde sich an den von Google zur Verfügung gestellten Praktiken orientiert. (Vgl. [**GoogleSRE**], S. 43f.)

3.4.2 Fehlerszenario A

In Fehlerszenario A wurde der Transaktions-Microservice so konfiguriert, dass er bei der Beantwortung einer Anfrage in 5% der Fälle einen internen Serverfehler meldete. Dieser injizierte Fehlerwert wurde gewählt, um eine ausreichend hohe Fehlerrate zu erwirken, ohne dabei in der initialen Canary Phase einen signifikanten Systemausfall zu provozieren. Die zu überprüfende Metrik im Hinblick auf das SLO, ist das Verhältnis der Summe fehlerhaft beantworteter Anfragen zur Summe aller Anfragen.

$$\text{Fehlerrate} = \frac{\sum \text{Anfragen}_{\text{fehlerhaft}}}{\sum \text{Anfragen}_{\text{gesamt}}} \tag{3.1}$$

(Vgl. [**GoogleSRE**], S. 38) Hierbei zählen als fehlerhafte Anfragen, diejenigen mit HTTP-Statuscode 500.

3.4.3 Fehlerszenario B

In Fehlerszenario B wurde der Transaktions-Microservice so konfiguriert, dass er bei der Beantwortung einer Anfrage zusätzlich zur normalen Antwortzeit 15ms warten musste. Diese Wartezeit wurde initial auf den doppelten SLO-Wert gesetzt, was während der erster Tests zu Systemausfällen führte, daher wurde der Wert insoweit nach unten korrigiert, ohne das SLO-Limit zu unterschreiten. Die zu

überprüfende Metrik ist die Antwortzeit über 95% aller Anfragen, die ohne Fehler beantwortet wurden.

$$\text{p95 Latenz} = \text{Perzentil}_{95}(L_{status=200}) \tag{3.2}$$

(Vgl. [**GoogleSRE**], S. 42) Wobei alle erfolgreichen Anfragen durch den HTTP-Statuscode 200 gekennzeichnet sind.

3.4.4 Analyseintervalle

Um ein einheitliches Analyse-Verfahren zu garantieren, wurden die Werte für die Analyseintervalle für alle Analysemethoden vereinheitlicht. Alle 60 Sekunden findet die Überprüfung der Metrik statt. Dieser Wert orientiert sich an der wissenschaftlichen Arbeiten (Vgl. [**islam2024predictive**], S. 244) und wurde zudem mit dem Standardwert des Scrape-Intervalls von Prometheus vereinheitlicht. (Vgl. [**PromDef**]) Während für die AnalysisTemplates ein Fehlerlimit von eins gesetzt wurde, was eine einmalige Überschreitung des Schwellwerts zulässt, wurde der Alertmanager ebenso konfiguriert, sodass er bei der ersten Schwellwertüberschreitung den Status *Pending* annimmt, demnach zulässt und erst bei der zweiten Schwellwertüberschreitung den alarmierten Zustand auslöst. Es wurde sich für eine Anzahl von zwei Überschreitungen der Schwellwerte entschieden, um *Flapping Alerts* auszuschließen. (Vgl. [**vinnakota2025creating**], S. 176)

Kapitel 4

Ergebnisse

Für die Fehlerszenarien wurden die Zeit bis zur Detektion und die Zeit bis zur Wiederherstellung des stabilen Zustands mithilfe der Kubernetes Events bestimmt. So wird eine möglichst genauer zeitlicher Wert der Start- und Endzeit von Injektion, über Fehlererkennung und Ende des Rollbacks sichergestellt. Es werden die Kubernetes Events als Referenz für das Ende des Rollbacks anstatt der Normalisierung der Prometheus Graphen verwendet.

4.1 Fehlerszenario A

4.1.1 Mittlere Detektionszeit und Mittlere Wiederherstellungszeit

Abbildung 4.1 zeigt, dass die Zeit bis zur Detektion für Rolling Updates im Vergleich zu den Canary Methoden mit rund 3,3 Minuten am geringsten war. Gefolgt von der Canary Methode mit isolierter Metrikanalyse mit rund 8 Minuten, am längsten benötigte die Canary Methode mit globaler Analysemetrik, hier betrug die Zeit bis zur Detektion im Schnitt 12 Minuten. Die geringste Wiederherstellungszeit erreichte die Canary Methode mit isolierter Metrikanalyse, im Durchschnitt betrug die Zeit rund eine halbe Minute. Die Canary Methode mit globaler Analysemetrik stellte den stabilen Zustand innerhalb von rund einer Minute wieder her. Das Rolling Update benötigte für die Wiederherstellung des stabilen Zustands fast zwei Minuten.

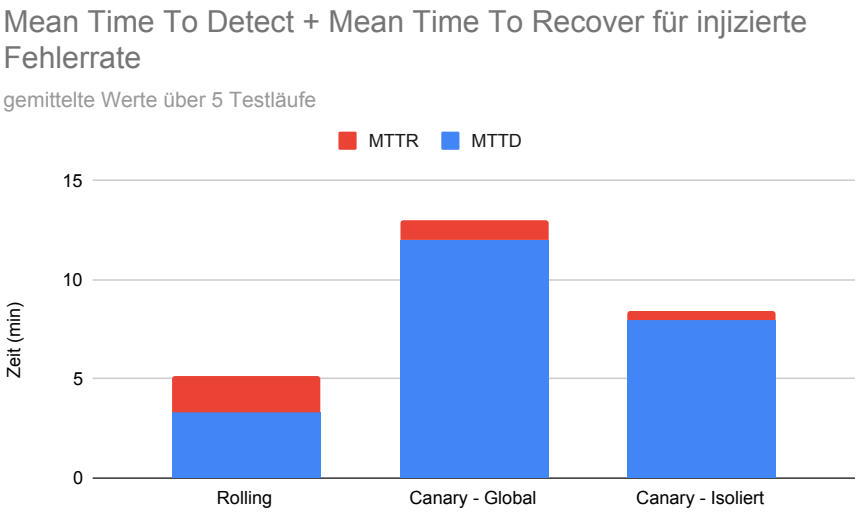

Abbildung 4.1: Summe der Zeit bis zur Detektion und der Zeit bis zur Wiederherstellung für injizierte Fehlerrate

4.1.2 Die einzelnen Testläufe im zeitlichen Verlauf

Die Diagramme 4.2, 4.3, 4.4 zeigen den globalen Einfluss der injizierten Fehlerrate auf das System.
Beginnend mit dem Vergleich der einzelnen Testläufe für das Rolling Update zeigt Abbildung 4.2, dass die Fehlerrate den Maximalwert von 5% innerhalb von 5 Minuten erreichte und dieser nach Abschluss des Rollbacks wieder auf null sank.

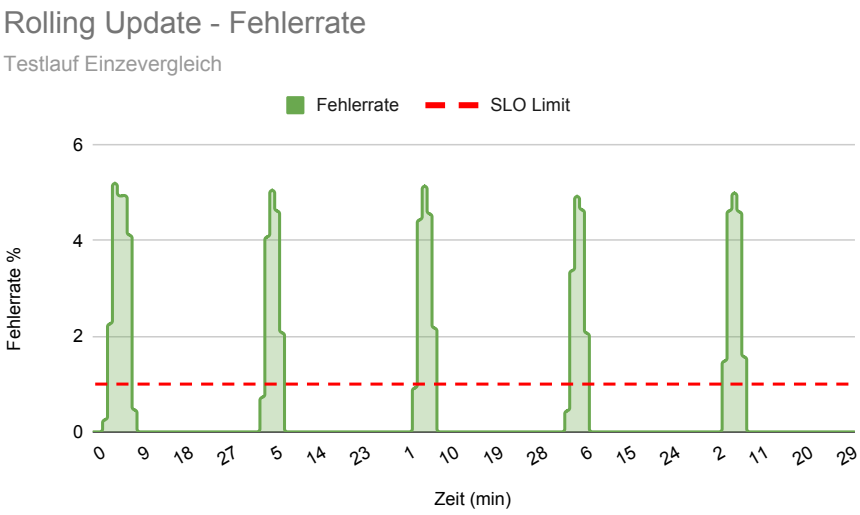

Abbildung 4.2: Fehlerrate Rolling Update im Einzelvergleich

In Abbildung 4.3 wird ersichtlich, dass die Canary Release Strategie mit globaler Metrikanalyse einen Maximalwert von 2,3% Fehlerrate innerhalb von 13 Minuten

erreichte. Auffällig ist der erste Testlauf, der nach elf Minuten seinen Peak von 1,1% Fehlerrate erreichte und im Gegensatz zu den folgenden Testläufen direkt zurückgerollt wurde. Inwiefern die Analyse hier schneller reagiert, wird später diskutiert.

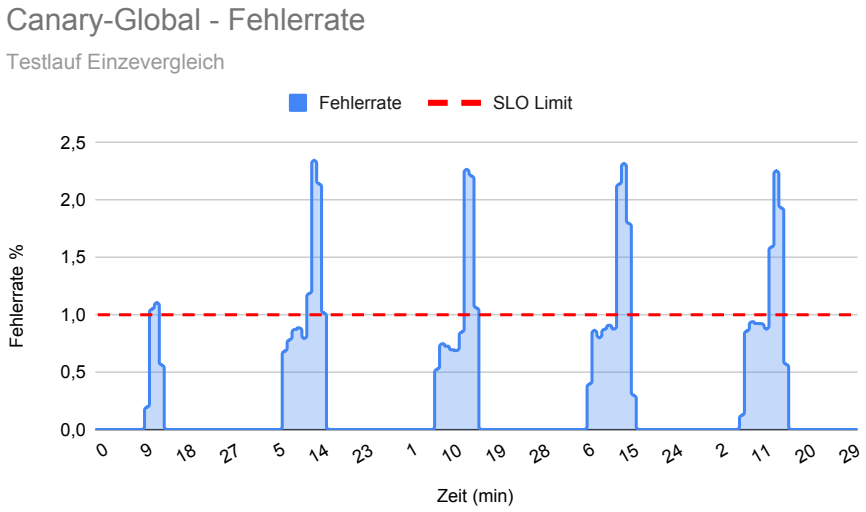

Abbildung 4.3: Fehlerrate Canary Release globaler Metrikanalyse im Einzelvergleich

Während beide vorherigen Deployment Strategien das SLO-Limit überschritten, zeigt Abbildung 4.4, dass die isolierte Analysemetrik das SLO-Limit erfolgreich über alle Testläufe eingehalten hat. Hier wurden die Maximalwerte im Schnitt innerhalb von neun Minuten erreicht.

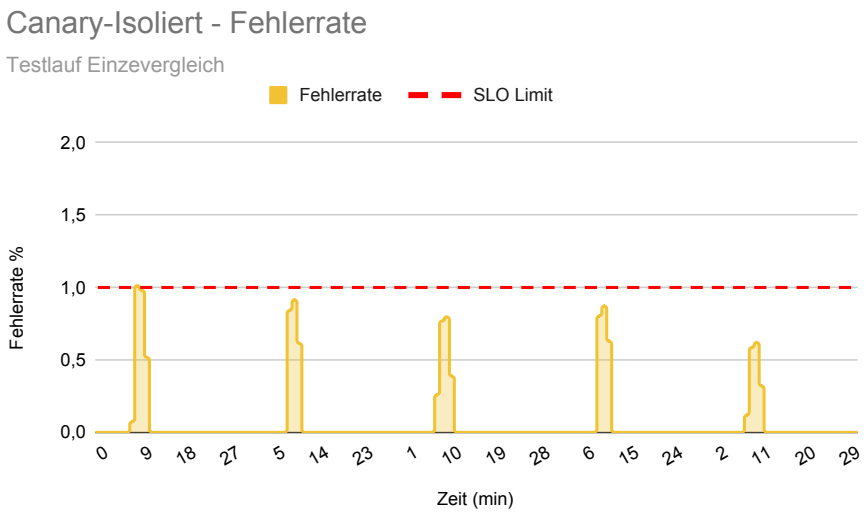

Abbildung 4.4: Fehlerrate Canary Release isolierter Analysemetrik im Einzelvergleich

4.1.3 Die gemittelten Werte der Deployment Strategien im Vergleich

Deutlicher werden die ermittelten Werte im direkten Vergleich der Deployment Strategien. In Abbildung 4.5 wird der Zusammenhang aus Zeit bis zur Detektion und Einfluss der injizierten Fehlerrate für die unterschiedlichen Deployment Strategien stärker ersichtlich, als bei der Einzelbetrachtung. Während das Rolling Update bereits den Wiederherstellungsprozess der stabilen Version abgeschlossen hatte, entstanden die Fehler der Canary Methoden erst. Dafür begrenzte sich der Einfluss der injizierten Fehlerrate bei den Canary Methoden, während das Rolling Update die vollen 5% injizierte Fehlerrate aufwies.

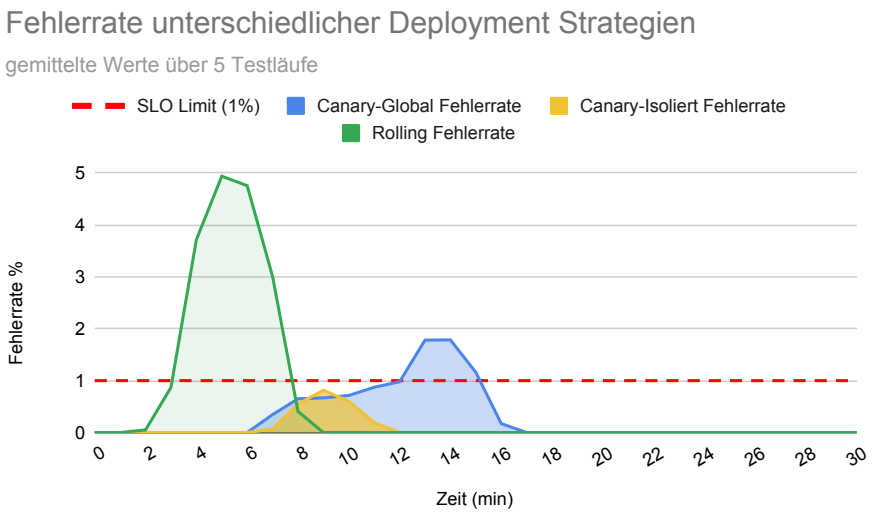

Abbildung 4.5: Fehlerrate unterschiedlicher Deployment Strategien

4.2 Fehlerszenario B

4.2.1 Mittlere Detektionszeit und Mittlere Wiederherstellungszeit

Abbildung 4.6 zeigt auch hier, dass das Rolling Update die geringste Zeit von der Fehlerinjektion bis zur Wiederherstellung des stabilen Zustands benötigte. Im Durchschnitt über alle Testläufe rund 6,9 Minuten. Das Canary Release mit isolierter Analysemetrik benötigte von der Injektion bis zur Wiederherstellung rund

7,3 Minuten, während das Canary Release mit globaler Metrikanalyse mit rund 13,3 Minuten am längsten benötigte.

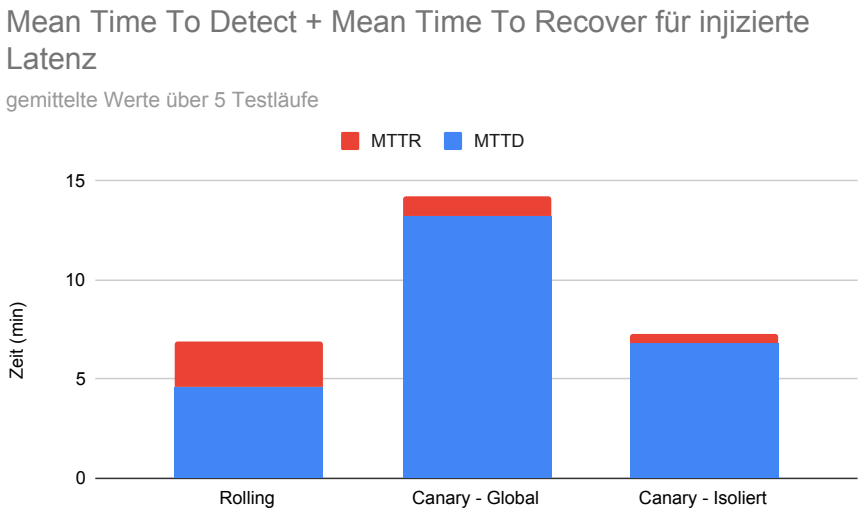

Abbildung 4.6: Summe der Zeit bis zur Detektion und der Zeit bis zur Wiederherstellung für injizierte Latenz

4.2.2 Die einzelnen Testläufe im zeitlichen Verlauf

Die Diagramme 4.7, 4.8, 4.9 zeigen den globalen Einfluss der injizierten Latenz auf das System. Beginnend mit dem Vergleich der einzelnen Testläufe für das Rolling Update zeigt Abbildung 4.7, dass die Latenz Maximalwerte von über einer Sekunde Antwortzeit betrug.

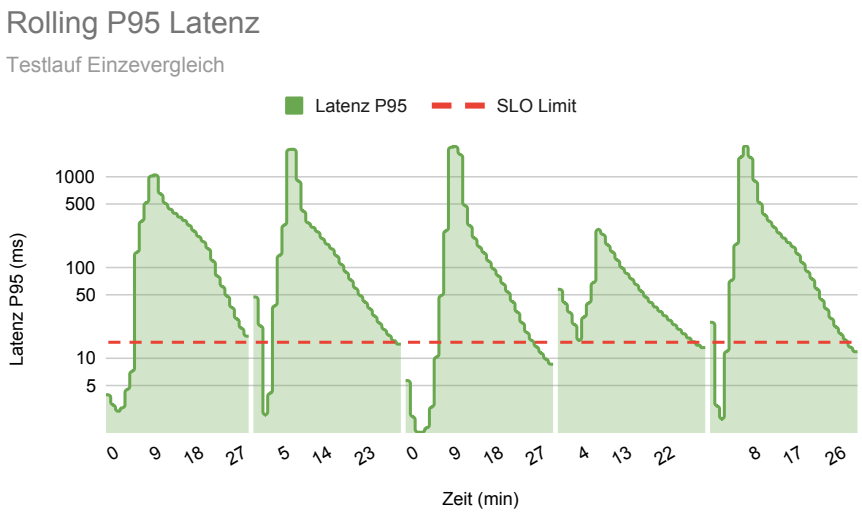

Abbildung 4.7: Latenz Rolling Update im Einzelvergleich

In Abbildung 4.8 wird ersichtlich, dass die Canary Release Strategie mit globaler Metrikanalyse einen Maximalwert von bis zu 290 Millisekunden an Antwortzeit benötigte. Dies entspricht einer Steigerung um den Faktor 20 der erlaubten Antwortzeit entsprechend des SLO.

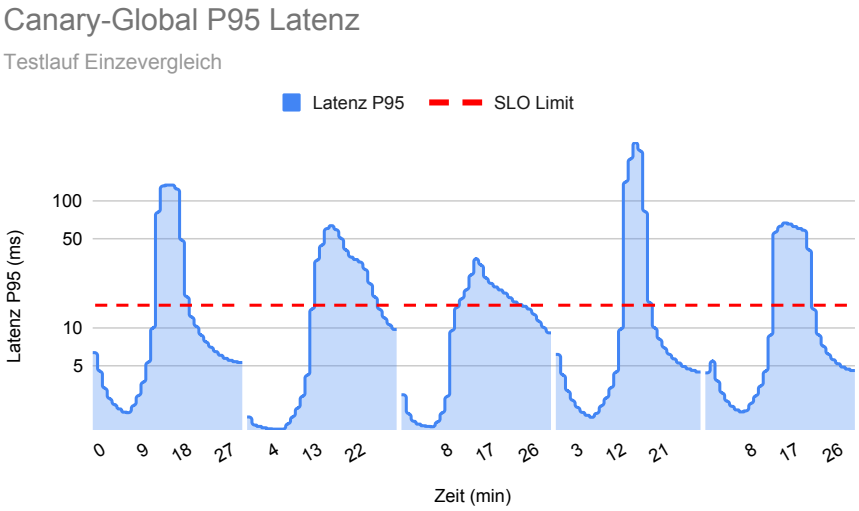

Abbildung 4.8: Latenz Canary Release globaler Metrikanalyse im Einzelvergleich

Die beiden vorherigen Deployment Strategien überschritten das SLO-Limit deutlich. Abbildung 4.9 zeigt, dass die isolierte Analysemetrik das SLO-Limit erfolgreich über alle Testläufe eingehalten hat. Auffällig sind im Diagramm einzig die Warm-Up Phasen während des Deploymentstarts.

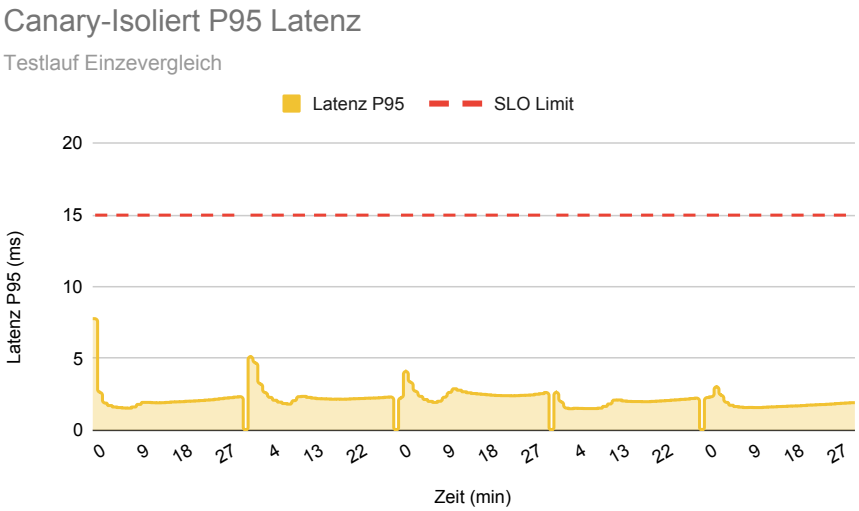

Abbildung 4.9: Latenz Canary Release isolierter Analysemetrik im Einzelvergleich

4.2.3 Die gemittelten Werte der Deployment Strategien im Vergleich

Der direkte Vergleich der Deployment Strategien bringt einen deutlichen Unterschied des isolierten gegenüber des globalen Analyseansatzes und der Analyse des Rolling Updates zum Vorschein. Die globale Analyse des Canary Releases und das Rolling Update überschritten das gesetzte Limit um das 6,67-fache und Respektive um das 66,67-fache. Während die isolierte Metrikanalyse des Canary Releases keinen Einfluss auf das Gesamtsystem aufweist.

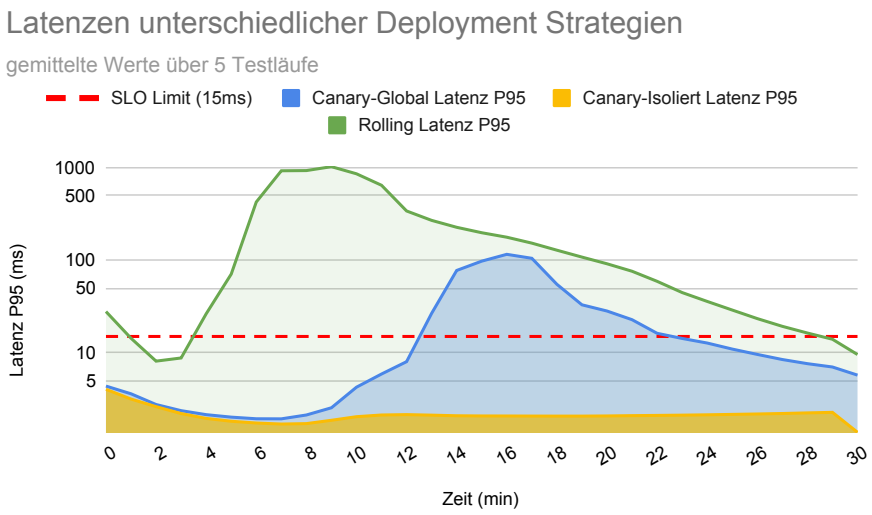

Abbildung 4.10: Latenzen unterschiedlicher Deployment Strategien

4.3 Fehlerbudget Verbrauch pro Deployment Strategie

4.3.1 Fehlerszenario A

Abbildung 4.11 zeigt den Einfluss der Analysemetrik und der Deployment Strategie auf den Verbrauch des Fehlerbudgets. Den geringsten Verbrauch erreichte die Canary Release Strategie mit isolierter Analyse. Der Bedarf betrug 7,4% des Gesamtbudgets. Das Canary Release mit globaler Analyse erreichte einen Bedarf von 30,42% am Gesamtbudget. Den höchsten Verbrauch des Fehlerbudgets bedurfte das Rolling Update mit 59,14%. In diesem speziellen Szenario verringerte

die Canary Release Strategie mit globaler Analysemetrik den Verbrauch des Fehlerbudgets im Gegensatz zum Rolling Update um 28,71% und die isolierte Analyse erreichte eine Verringerung des Budgetverbrauchs gegenüber dem Rolling um 51,72%. Weiter ist eine Verringerung des isolierten gegenüber des globalen Ansatzes um 23,01% festzuhalten.

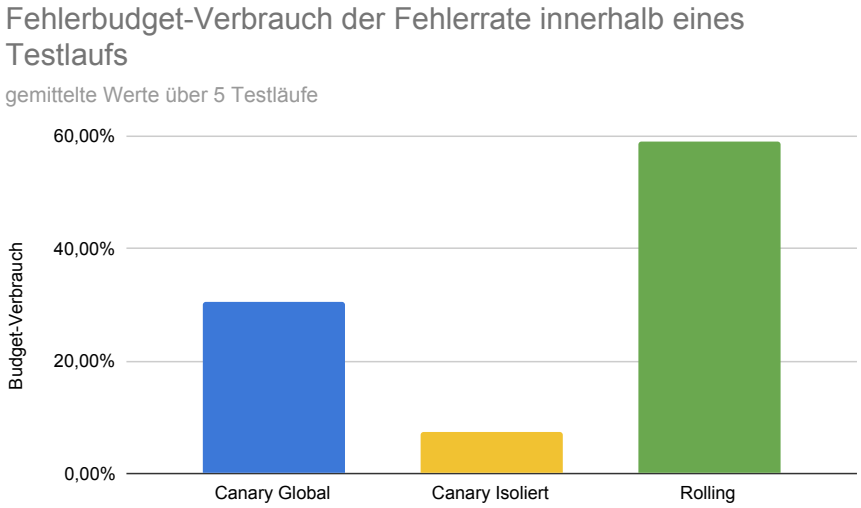

Abbildung 4.11: Verbrauch des Fehlerbudgets unterschiedlicher Deployment Strategien bei injizierter Fehlerrate über die Testdauer

4.3.2 Fehlerszenario B

Abbildung 4.12 zeigt ebenfalls den Einfluss der Analysemetrik und Deployment Strategie hinsichtlich des Verbrauchs des Fehlerbudgets. Während der geringste Bedarf von 0,25% am Fehlerbudget durch die isolierte Analysemetrik des Canary Releases erreicht wurde, verbrauchte die globale Analyse 2,59%. Das Rolling Update stellte mit rund 26% den höchsten Bedarf am Fehlerbudget. Die Einsparung des Fehlerbudgets von Canary Release mit globaler Analyse gegenüber dem Rolling Update beläuft sich auf 23,4%. Das Canary Release mit isolierter Analyse erreichte eine Einsparung von 25,74% gegenüber dem Rolling Update. Die Verringerung der Canary Release Strategie mit isolierter gegenüber globaler Analyse beläuft sich auf 2,34%.

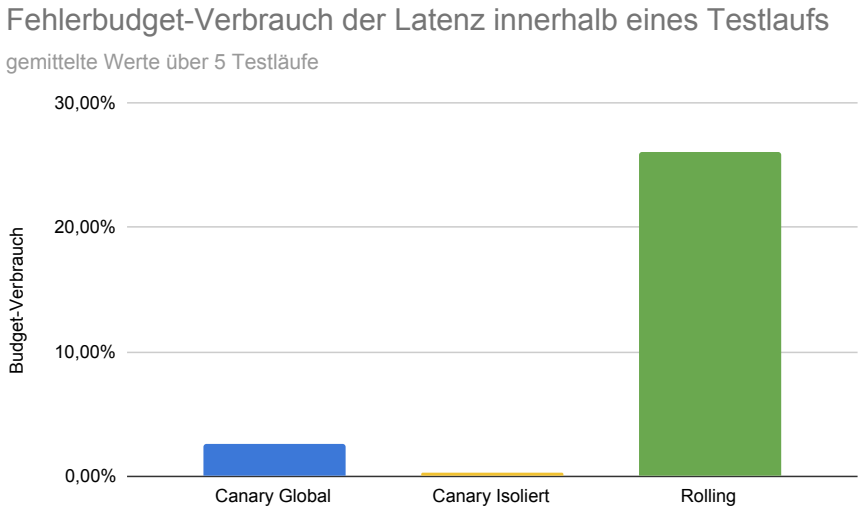

Abbildung 4.12: Verbrauch des Fehlerbudgets unterschiedlicher Deployment Strategien bei injizierter Latenz über die Testdauer

Kapitel 5

Diskussion

5.1 Interpretation der Ergebnisse

5.1.1 Fehlerszenario A

MTTD, MTTR

Die zeitlichen Unterschiede für das Diagramm 4.1 lassen sich mit dem Versuchsaufbau erklären. Bei den Rolling Updates wurde der Fehler sofort erkennbar, da 5% aller Anfragen an das System fehlschlugen. Der zeitliche Unterschied zwischen den Canary Release Methoden lässt sich mit der verwendeten Analysemetrik begründen. Für die globale Aggregation musste die Gewichtung des Datenverkehrs erst einen signifikanten Schwellwert überschreiten, dass die Fehler nicht von den fehlerfreien Pods überlagert wurden. Die isolierte Fehlererkennung stellte hingegen das direkte Verhältnis der fehlerhaften Anfragen zu den Gesamtanfragen an die Canary Version gegenüber und konnte daher einen zeitlichen Vorteil erlangen. Diskutiert wird später dennoch, weshalb während der initialen 10% Gewichtung keine Fehler erkannt wurden, obwohl diese mathematisch Vorkommen sollten.
Die Wiederherstellungszeiten lassen sich in Zusammenhang mit der Anzahl an Pods bringen, die ausgetauscht werden mussten, um die fehlerfreie Version wieder lauffähig zu machen. Die Rolling Updates mussten der Reihe nach alle Pods ersetzen, während bei den Canary Release Methoden noch fehlerfreie Pods lauffähig waren. Der Zeitunterschied von isolierter und globaler Analysemetrik lässt

sich weiter in Zusammenhang mit den fehlerfrei laufenden Pods bringen. Die Canary Methode mit isolierter Metrikanalyse detektierte die Fehler im zweiten Schritt, hier waren 50% User-Traffic auf die neue Version umgeleitet und die fehlerfreie Pod Anzahl betrug zwei. Die Canary Methode mit globaler Analysemetrik detektierte den Fehler erst im dritten Schritt mit 75% User-Traffic, die fehlerfreie Pod Anzahl betrug eins.

5.1.2 Fehlerszenario B

MTTD, MTTR

Wie auch für das Fehlerszenario A lassen sich diese zeitlichen Unterschiede im Diagramm 4.6 mit dem direkten Einfluss des injizierten Fehlers für das Rolling Update erklären. Gleich gilt auch der Vorteil der isolierten Analyse der Canary Releases gegenüber dem globalen Ansatz, da für die isolierte Metrik, der direkte Einfluss der Canary Version überprüft wurde, hingegen musste bei der globalen Metrikanalyse erst eine ausreichend hohe Gewichtung des Datenverkehrs auf die Canary Version erreicht werden, damit der Fehler erkannt wurde.

Bei der Betrachtung injizierter Fehlerrate konnte die Canary Strategie mit globaler Metrik-Aggregation die Zeit bis zur Wiederherstellung des stabilen Zustands von 1,8 auf 0,9 Minuten halbieren. Die Canary Strategie mit isolierter Metrik-Aggregation konnte diese Zeit wiederum um die Hälfte auf 0,4 Minuten reduzieren. Ähnliche Werte wurden für die Wiederherstellungszeit bei injizierter Latenz beobachtet. Hier reduzierte sich die Zeit von der Erkennung des Defekts bis zum stabilen Zustand von 2,2 Minuten für die Rolling Strategie auf 1,0 Minuten für die Canary Strategie mit globaler Aggregation und die Canary Strategie mit isolierter Aggregation reduzierte die Zeit wiederum auf 0,4 Minuten. Ähnliche Ergebnisse zur Reduktion der Wiederherstellungszeit des stabilen Systems finden sich auch in den Arbeiten von ([**william2025deployment**]; [**palvai2026designing**], S. 127; [**mohammed2021enhancing**])

Einzeltestläufe im zeitlichen Verlauf

Die massive Überschreitung des SLO-Limits für Rolling Updates ist mit blockierenden Threads zu erklären. Die Softwareversion hat es nicht geschafft die Anfragen in genügend geringer Zeit zu beantworten und die Pods wurden durch *OOMKills* kontinuierlich neu gestartet. Diese Unerreichbarkeit der Services ist als Downtime im Sinne des Totalausfalls zu bewerten. Eine Erklärung dieser Überlastung der Ressourcen kann der programmatischen Implementierung zugeschrieben werden. *Thread.sleep(VALUE)* führt dazu, dass die Prozesse auf der Hardware anhaltend blockieren und der Arbeitsspeicher durch sich immer weiter stapelnde Anfragen so stark gefüllt wird, dass es zum Out-Of-Memory Kill kommt.

5.2 Der erste Testlauf der injizierten Fehlerrate für das Canary Release mit globaler Metrikanalyse

Beim Erstellen der Graphen für die einzelnen Testläufe war dieser direkt auffällig. Offensichtlicher wurde die Auffälligkeit nach der Erstellung der Analyse der Canary Releases mit isolierter Metrik. Die Werte stimmten beinahe überein. Von der Erklärung abgesehen, dass die Messwerte hier zufällig innerhalb von zwei Minuten den Schwellwert geringfügig überschritten, wird vermutet, dass die Beachtung der JVM-Warmup-Zeiten eine Verbesserung der Detektionszeit hervorgerufen hätte, in dem Sinne, dass die Analyse mit einer initialen Verzögerung Fehler schneller detektiert hätte, da die ersten Pods bereits lauffähig und ansprechbar gewesen wären.

5.3 Nicht gemessene Fehler während der ersten Canary Phase

Bei der Analyse der Daten ist aufgefallen, dass praktisch keine Fehlerrate während der initialen Canary Phase, der Gewichtung des Datenverkehrs von 10%, erfasst wurde. Mathematisch betrachtet, sollten mit 1200 Anfragen pro Sekunde, einer Gewichtung von 10% und einer Fehlerrate von 5%, sechs Fehler pro Sekunde und dementsprechend 360 Fehler pro Minute auftreten, jedoch erfasste

weder die globale, noch die isolierte Analyse einen einzigen Fehler für die Gewichtung während der Zeit von fünf Minuten. Ein möglicher Grund für das Nicht-Erfassen der globalen Analyse dieser Fehlerrate kann statistische Irrelevanz sein. 360 Fehler zu 72.000 Anfragen pro Minute ist für die Aggregation der Metrik möglicherweise nicht signifikant genug. Andernfalls erklärt das die Nicht-Erkennung der isolierten Analyse nicht, da hier 360 Fehler zu 7.200 Anfragen stehen, was genau der injizierten Fehlerrate entspricht. Eine genaue Erklärung der Nicht-Erkennung lässt sich aus meiner Sicht nicht treffen.

5.4 Detektionszeit in Zusammenhang des Fehlerbudgetverbrauchs

Die Berechnung der Zeit bis zur Detektion des Fehlers zeigt deutlich, dass die Rolling Updates Analyse den Fehler am schnellsten erkannt hat. Jedoch hat diese Deployment Strategie auch den größten Verbrauch am Fehlerbudget. Wie im oberen Abschnitt beschrieben, wurden für die initiale Phase des Canary Releases jedoch kaum bis keine Defekte der Softwareversion festgestellt. Würde die Zeit der initialen Phase von der Detektionszeit abgezogen werden, so hätte die Canary Release Strategie mit isolierter Analyse eine geringere Detektionszeit gegenüber dem Rolling Update. Andersherum hätte die Datenverkehr Gewichtung innerhalb der initialen Phase auch schneller erhöht werden können, um einen statistisch signifikanten Schwellwert zu überschreiten und schneller Fehler zu provozieren. Letztlich zeigen die Daten der Detektionszeit im Zusammenhang mit dem Fehlerverbrauch, dass eine schnellere Detektionszeit nicht direkt mit einer Ersparnis des Fehlerbudgets einhergehen, sondern dass der Anteil betroffener Nutzer den entscheidenden Anteil am Verbrauch des Fehlerbudgets trägt.

Kapitel 6

Kritische Auseinandersetzung

Während diese Arbeit selbstverständlich mit hohen Ansprüchen im Bezug auf Best-Practices und der möglichst vollständigen Betrachtung der Forschungsfrage erstellt wurde, hat sie jedoch auch Grenzen. Zuerst sind hier die Hardware-Ressourcen als limitierender Faktor zu nennen. Zwei VM's mit jeweils 2 virtuellem CPU und acht Gigabyte RAM sind leistungsstark, jedoch einzeln unter dem Niveau eines Office-Rechners niedriger Preisklasse. ([**MM**]) Ein weiterer Kritikpunkt der Nodes ist die Wahl von *flex*-Instanzen, welche eine variable CPU-Leistung besitzen. Dies könnte zur Drosselung innerhalb intensiver Test-Tage geführt haben und wurde nicht berücksichtigt, die Testergebnisse legen dies jedoch nicht nah.
Am Versuchsaufbau kann ebenfalls Kritik geübt werden. Durch die Realisierung der Lasttests mithilfe Cluster-interner DNS Auflösung wurde eine Ebene des Netzwerkflusses (nach 2.1) und damit eine mögliche Fehlerquelle entfernt. Weitere Kritik kann an der Einfachheit der Systemarchitektur geübt werden. Dies stellt nur in sehr geringem Maße eine reale Microservice Architektur dar, wobei Latenz- und Fehlerraten dann durch kaskadierende Effekte entstehen. (Vgl. [**yi2015explaining**])
Dies lässt jedoch Spielraum für weiterführende wissenschaftliche Arbeiten. Am Versuchsaufbau sind die Canary-Steps kritisch zu betrachten. Der Sprung von 10 auf 50 Prozent Datenverkehr-Gewichtung ist sehr groß und da festgestellt wurde, dass während der initialen Phase keine Fehler erkannt wurden, sollte hier eine feinere Gewichtung gewählt werden.

6.1 Empfehlungen für weiterführende Forschungen

Mit den Erkenntnissen dieser Arbeit wäre weiterführende Forschung im Hinblick auf den Einfluss der feineren Gewichtung für die Detektionszeit spannend. Zusätzlich kann untersucht werden, inwiefern sich kürzere Analyseintervalle auf die Ergebnisse dieser Forschung auswirken oder inwiefern sich Deployments mit Blue-Green Strategie unterscheiden. Als abschließende Forschung würde sich eine Analyse des Kostenfaktors, der durch den Mehraufwand dedizierter Monitoring Nodes entsteht anbieten, welche in das Verhältnis zum ersparten Fehlerbudget gesetzt wird.

Kapitel 7

Fazit

Die Arbeit untersucht den Zusammenhang agiler Softwareentwicklung mit schnellen Implementierungszyklen und der notwendigen Stabilität während des produktiven Betriebs. Das Fehlerbudget dient als Balance zur Erreichung beider Ziele. Dieses definiert, wie viel Risiko während der Innovation eingegangen werden kann, ohne die Stabilität des Systems maßgeblich zu verschlechtern. Im Zentrum der Arbeit steht die Forschungsfrage, *In welchem Maße kann durch begrenzten Nutzerzugriff und die Verwendung von White-Box Monitoring der Einfluss fehlerbehafteter Softwareversionen auf ein Fehlerbudget beeinflusst werden?* Die Beantwortung der Frage wurde in einem speziellen Versuchsaufbau realisiert. Dabei wurden die zu untersuchenden Services einer Last von 1200 Anfragen pro Sekunde ausgesetzt. Anschließend wurde eine Fehlerrate von fünf Prozent und im Folgeszenario eine erhöhte Antwortzeit in einen der Services injiziert, während der andere Service als Referenzwert für etwaig auftretende infrastrukturelle Fehler diente. Die Untersuchung konzentrierte sich auf zwei verbrauchsverringernde Mechanismen, die Begrenzung des Nutzerzugriffs mithilfe der Canary Bereitstellungsstrategie und der Verringerung der Detektionszeit durch White-Box Monitoring in der ausgerollten Canary Version. Die Ergebnisse dieser Arbeit zeigen, dass für diesen speziellen Versuchsaufbau mithilfe der Begrenzung des Nutzerzugriffs eine Verringerung des Fehlerbudgets von 28% erreicht wurde. Mithilfe von White-Box Monitoring, bzw. isolierter Metrik-Aggregation der Canary Version konnten weitere 23% des Fehlerbudgets geschützt werden, was sich zu einer Gesamtverringerung von 51% addiert. Für die zusätzlich injizierte Latenz konnte durch die Begrenzung des Nutzerzugriffs 23% des Fehlerbudgets gespart werden, während die isolierte Metrik-Aggregation weitere 2,3% am Budget einsparen

konnte und somit eine Gesamtverringerung von 25% erreichen.

Anhang

Eigenständigkeitserklärung

Hiermit erkläre Ich, dass die vorliegende Arbeit selbstständig und nur unter Verwendung der angegebenen Literatur und Hilfsmittel angefertigt habe. Stellen, die wörtlich oder sinngemäß aus Quellen entnommen wurden, sind als solche kenntlich gemacht. Diese Arbeit wurde in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt.

Borna, den 24.03.2026 Paul Fuchs

Implementierung

Die Implementierung, der Testaufbau und alle Logs der Testdurchläufe dieser Arbeit sind unter `https://github.com/Vulnerius/Bachelorthesis-impl/tree/thesis-final` verfügbar.